

Introduction au C4 Model

→ Le modèle C4 a été créé par **Simon Brown** entre 2006 et 2011 pour remédier aux carences de l'UML et moderniser la visualisation de l'architecture logicielle.

Contrairement aux approches traditionnelles souvent jugées trop complexes, le C4 se veut une technique **légère et pragmatique**.

→ Il repose sur l'idée que le développement logiciel devrait être traité comme une discipline d'ingénierie où la communication visuelle est essentielle.

C'est un modèle qui privilégie les **abstractions communes** plutôt qu'une notation rigide, permettant aux équipes de s'accorder sur le vocabulaire technique (systèmes, conteneurs, composants) avant de se soucier de la forme des boîtes et des flèches.

Pourquoi le C4 marche bien en entreprise

- Impose une progression logique
 - "De quoi parle le système ?" → "De quoi est-il composé ?" → "Comment c'est structuré à l'intérieur ?".
 - Orienté "lecteur"
 - chaque niveau répond à une question différente et cible un public différent.
 - Réduit les débats stériles de notation UML
 - l'objectif est la compréhension, pas la perfection graphique.
 - Itératif
 - On peut commencer avec 1 diagramme (contexte) et enrichir progressivement.
-

Pourquoi utiliser le C4 Model ?

L'utilisation de ce modèle présente plusieurs avantages majeurs identifiés dans les sources :

- **Réduire le "fossé modèle-code"** : Le C4 assure que les éléments dessinés sur les diagrammes correspondent à des réalités concrètes dans le code source. Si un "Composant de Logging" est sur le diagramme, il doit exister physiquement dans le code.
- **Navigation hiérarchique (Mantra de Shneiderman)** : Le modèle permet de raconter une histoire à différents niveaux de détail : "Vue d'ensemble d'abord, zoom et filtrage, puis détails à la demande".

- **Adaptation à l'audience :**

- Les niveaux supérieurs (Contexte) s'adressent aussi bien aux décideurs **non techniques** qu'aux techniciens.

- Les niveaux inférieurs (Conteneurs, Composants) sont destinés aux développeurs et aux équipes opérationnelles pour comprendre les choix technologiques et la distribution des responsabilités.

- **Améliorer l'on-boarding :** Il sert de "carte" pour aider les nouveaux développeurs à naviguer dans une base de code complexe.

- **Éviter l'encombrement :** En séparant les niveaux de détail, on évite les diagrammes illisibles où des milliers de classes s'entrecroisent.

Comment utiliser le C4 Model ?

La technique consiste à créer quatre niveaux de diagrammes, chacun zoomant davantage sur le système :

1. Niveau 1 : Diagramme de Contexte Système La vue « argumentaire éclair »

- **Question :** "Quel est le système, pour qui et avec quoi il interagit ?"

- **But :** Montrer comment le système s'insère dans le monde environnant.

- **Éléments :** Le système lui-même (une boîte centrale), les **utilisateurs** (personnes) et les **systèmes externes** avec lesquels il interagit.

- **Focus :** Relations et périmètre, pas de détails techniques.

- **Contenu attendu :**

- Le "Software System" au centre (le produit)

- Les personnes (acteurs) qui l'utilisent

- Les systèmes externes (paiement, SMS, labo, SI existant, etc.)

- Les relations principales (qui parle à qui, pour quoi)

- **Public :** Tout le monde (parties prenantes, chefs de produit, utilisateurs).

Ce diagramme montre votre système comme une seule boîte. Il se concentre sur le qui et le quoi. Qui utilise le système ? Avec quels systèmes externes (comme un fournisseur LLM) communique-t-il ? Il ignore tous les détails techniques de mise en œuvre.

2. Niveau 2 : Diagramme de Conteneurs - La vue « planification des infrastructures »

- **Question :** "En quoi le système est-il découpé à haut niveau, et comment ces morceaux communiquent ?"

- Un "container" ici n'est pas forcément Docker : c'est une unité déployable/exécutible ou un stockage significatif (web app, mobile app, API, service, database...).

- **But :** Décomposer le système en ses grandes unités techniques.

- **Définition d'un conteneur :** Une application ou une banque de données séparément déployable (ex: application web, base de données, application

mobile, microservice).

- **Focus** : Choix technologiques et communication entre conteneurs.
- **Contenu attendu** :
 - Applications (web, mobile, back-office)
 - API / services / jobs
 - Bases de données / files / caches
 - Protocoles/technos importantes (HTTP, gRPC, MQ, SQL...)
 - Relations entre conteneurs

- **Public** : Architectes, Développeurs, DevOps.

Maintenant, zoomons. Un « conteneur » est une unité déployable séparément. C'est ici que nous voyons votre pile technologique moderne en action. Nous montrons l'application Web, les microservices, les files d'attente SQS, la base de données relationnelle et la base de données vectorielle. Il montre comment les données circulent entre ces principaux blocs de haut niveau.

3. **Niveau 3 : Diagramme de Composants- La vue « équipe de développement »

- **Question** : "À l'intérieur d'un container, quels composants structurent le code et les responsabilités ?"
 - Un "component" = un bloc logique cohérent (Controller, Service applicatif, Repository, Adapter, etc.), selon le style architectural (Clean/Hexa, Layered, DDD...).

- **Contenu attendu** :

- Composants et responsabilités (phrases courtes)
- Interfaces majeures (API internes, ports/adapters)
- Dépendances entre composants

- **But** : Zoomer à l'intérieur d'un conteneur pour montrer ses composants internes.

- **Définition d'un composant** : Un regroupement de fonctionnalités liées derrière une interface propre, s'exécutant dans le même processus que le conteneur.

- **Public** : Développeurs.

En zoomant sur un micro-service spécifique – disons le service de requête AI – nous voyons ses composants internes. Nous voyons le générateur d'intégration, le Vector Retriever et le LLM Prompt Builder. Cela aide un développeur à comprendre les responsabilités internes du code.

4. Niveau 4 : Diagramme de Code - La vue "implémentation"

- **Question** : "Quelle est la structure détaillée du code ?"
 - Souvent inutile en documentation d'architecture : le code et l'IDE font déjà ce travail. Utile uniquement pour expliquer un pattern complexe, une partie critique, ou pour un enseignement très "design".
- **But** : Montrer l'implémentation d'un composant (souvent via un diagramme de classes UML).

- **Note** : Ce niveau est souvent considéré comme **optionnel**, car les détails se trouvent déjà dans le code.
 - **Public** : Développeurs.
C'est le niveau des classes et des objets.
Dans le monde moderne, nous dessinons rarement cela manuellement ; nous laissons généralement le code être la source de vérité ou le générons automatiquement si nécessaire.
-

Ce que C4 ne cherche pas à faire

- C4 ne remplace pas tous les artefacts d'architecture.
- Il ne décrit pas en détail le comportement à l'exécution.
- Il ne modélise pas la topologie de l'infrastructure ni les règles de sécurité réseau.
- Il ne supprime pas le besoin de diagrammes de séquence ou d'ADR.

Ce qu'il apporte, en revanche, c'est une ossature structurelle stable — un modèle mental partagé sur lequel les autres diagrammes peuvent s'appuyer.
